CH2: Common Mistakes in Declarations and Definitions

1. Variables must be declared to the compiler before being used.

- In the C language, before using a variable, it is necessary to declare its type and name.

For example:

```
int number; // The compiler now knows that there is an int called "number".
```

As we know:

• Declaration → "This name and type exist, defined elsewhere."

• Definition → "Allocate memory for this name."

```
int number; // definition (both declared and allocated)
extern int number; // declaration (defined in another file, just declaring here)
```

2. How to participate in the compilation?

main.c

```
#include <stdio.h>
extern int counter;  // declaration

int main() {
    counter = 5;
    printf("Counter = %d\n", counter);
    return 0;
}
```

lib.c

```
int counter = 0;  // definition
```

And output will be:

```
Counter = 5
```

NOTE: In the linker phase, it combines the request "counter is needed but it is not defined" in main.o with the definition "counter is here" in lib.o.

3. What if counters are defined in two different files?

lib1.c

```
int counter = 0;  // definition
```

lib2.c

```
int counter = 10; // the second definition
```

Then typing the code below in the terminal:

" gcc main.c lib1.c lib2.c -o program "

Here you'll get a linker error: multiple definition of `counter`

Because in C, global (external) variables can only be defined in one place.

Solution: use extern

Correct method:

• Definition in only one file:

```
int counter = 0;  // lib.c
```

Declaration in all other files:

```
extern int counter; // main.c, other files
```

This way, everyone shares the same global variable.

4. Type Conversion in C

In C, when different data types (e.g., int, float, char) are used together, the compiler **automatically converts them to the same type**.

- Smaller types are promoted to larger types.
- The operation is always performed according to the **largest type**.

However, during conversion:

- Decimal parts may be lost.
- Higher bits can be truncated.
- The sign bit may be extended.
- Performance may decrease (especially with float and double).

Example:

```
int a = 5;
float b = 2.5;
```

When doing a + b, the compiler cannot operate directly because int and float are stored in completely different formats.

Could this cause a memory bottleneck?

Yes, it can.

In C, smaller types (char, short, int) are promoted to larger types (int, float, double) during operations. This may cause: loss of decimals, bit truncation, or slower performance.

In embedded systems (limited RAM/CPU):

- char is promoted to int, creating overhead.
- float is often processed as double, which is slower.

Recommendations:

- Use **int** instead of **char** for efficiency.
- Use **double** instead of **float** for better performance.

5. Important Notes on Implicit Conversions in C

a. Automatic conversion is not always safe.

Promoting smaller → larger (e.g., int → double) is usually fine.

But larger → smaller may cause **data loss**.

b. Performance overhead.

Every conversion happens at runtime.

char and short are promoted to int.

float is often processed as double.

Extra CPU cycles can slow down embedded systems.

c. Unexpected results.

Implicit conversions may change program logic, especially with signed vs. unsigned types.

Example: -1 < 4000000000U evaluates as false because -1 is converted to an unsigned value.

Additional Note

- Use **explicit casts** ((int)x, (float)y) to avoid surprises.
- With explicit cast, the compiler assumes you accept potential data loss.
- Responsibility lies with the programmer; unexpected behavior is avoided.

5. Indirect Addressing

Why Write to Specific Memory Locations?

- **Dynamic memory**: Pointers + malloc let you allocate only what is needed, avoiding waste.
- **Without pointers**: You only handle copies → cannot change originals or access hardware.
- **With pointers**: You can update real values, manage memory efficiently, access hardware addresses, and build complex structures.

Conclusion:

- **Use copy** when data safety is important (temporary results, no changes and safety).
- **Use pointer** when updates, flexibility, performance, or hardware access are required.

8-bit CPUs read/write 1 byte at a time, while **16-bit CPUs** handle 2 bytes in one step, making memory access faster.
In 16-bit systems, integers start at even addresses (low byte at odd, high byte at even).
Designing a 4-bit CPU would be inefficient because of complex memory paths and limited ALU capacity.

Feature	8-bit CPU	16-bit CPU
Data Bus	8 bit (1 byte)	16 bit (2 bytes)
Read/Write	1 byte at a time	2 bytes at a time
Memory Addressing	Single byte	Even addr → High byte Odd addr → Low byte
Performance	Slower	Faster
Examples	8051, Z80	8086, Motorola 68000
Limitation	Slow for large data	More efficient but needs more hardware

Therefore,

8-bit → 1 byte/cycle,

16-bit → 2 bytes/cycle → higher speed and efficiency.